K. ความรู้จาก Quizizz ในห้องเรียน (4 ชุด) — จัดเรียงตามชุด

📖 รู้จักส่วนนี้ก่อนอ่าน

ความหมาย: ความรู้ที่สกัดจากแบบควิซในห้องเรียนจริง 4 ชุด (Signal/Pipe, Shared Memory (หน่วยความจำที่แชร์กัน), Semaphore, File Management) — ทุกข้อมี "ความรู้สำคัญ" ที่ตอบโจทย์แบบเดิมได้ทันที

ความสำคัญกับข้อสอบ: นี่คือธนาคารข้อสอบย่อที่ตรงสไตล์อาจารย์ที่สุด — หลายข้อมีกับดักที่อาจารย์ตั้งใจหลอก (เช่น ข้อ 8 ชุด 4 มีคีย์ขัดตำรา — ต้องตอบ False ตามคีย์!) อ่านก่อนสอบคุ้มที่สุด

ใช้จริงในคอมพิวเตอร์: ประเด็นในควิซคือข้อผิดพลาดจริงที่โปรแกรมเมอร์พลาดบ่อย เช่น เข้าใจว่า signal() ใช้ส่งสัญญาณ, ลืมปิดปลาย pipe (ท่อส่งข้อมูลทางเดียว) — รู้ไว้เขียนโค้ดจริงไม่พัง

ทุกข้อเป็นความรู้ที่ตอบโจทย์แบบเดิมได้ทันที — อ่านกล่อง "ความรู้สำคัญ" ต่อข้อ

K1. Quiz 1 — IPC & Unix Signal

ข้อ 2 False

"Signal ใช้ส่งข้อความระหว่าง process"

ความรู้: Signal = **event notification** เท่านั้น — ไม่มี payload แนบข้อมูลไม่ได้ ผู้รับรู้แค่ว่า "signal ชนิด X มา" · ส่งข้อมูลจริงใช้ Message Passing (การส่งข้อความผ่าน OS) / Shared Memory

ข้อ 3 Message Passing

"IPC ที่ OS จัดจังหวะการรับส่งให้"

ความรู้: Message passing ให้ OS จัดทั้ง send/receive และ**จังหวะ** (blocking/non-blocking) · Shared Memory จัดเตรียมแค่พื้นที่ — **ซิงก์จังหวะเป็นหน้าที่ของ process เอง** (เช่นใช้ semaphore)

ข้อ 4 SIGINT

"Ctrl-C ส่ง signal อะไร"

ความรู้: Ctrl-C → **SIGINT** (INT = INTerrupt) · คู่เทียบ: SIGFPE = หารศูนย์, SIGTERM = ขอจบสุภาพ (default ของ kill (ส่งสัญญาณไปยังกระบวนการอื่น)), SIGQUIT = Ctrl-\, SIGKILL = ฆ่าทันทีจับไม่ได้ (9)

ข้อ 5 False

"signal() ใช้ส่ง signal ระหว่าง process"

ความรู้: `signal()` = **ลงทะเบียน handler (ฟังก์ชันรับมือเหตุการณ์)** ("โดน X แล้วทำอะไร") · ตัวส่งคือ `kill(pid, sig)` (ส่งข้าม process) และ `raise(sig)` (ส่งให้ตัวเอง)

ข้อ 6 False

"1 process จัดการ signal ได้แค่ 1 ตัว"

ความรู้: จัดการได้**หลาย signal** — แต่ละชนิดมี handler ของตัวเอง · ข้อจำกัดจริง: 1 ชนิด มี 1 handler (ลงทะเบียนซ้ำ = ทับตัวเดิม)

ข้อ 7 False

"kill() ใช้ส่งได้แค่ SIGKILL"

ความรู้: System (ระบบ) call `kill(pid, sig)` ส่งได้**ทุกชนิด** — SIGKILL แค่ 1 ตัวเลือก (ชื่อมาจากประวัติศาสตร์ ทำให้เข้าใจผิด)

ข้อ 8 True

"คำสั่ง kill ใน UNIX ใช้ส่ง signal ให้ process"

ความรู้: หน้าที่จริงของคำสั่ง kill = **ส่ง signal** (default SIGTERM) · `kill -9 pid` = SIGKILL · ⚠️ แยกจากข้อ 7: system call ส่งได้ทุกชนิด (โจทย์ใส่ "เท่านั้น" มาหลอก)

K2. Quiz 2 — Shared Memory & Message Passing (Pipe)

ข้อ 2 False

"Message passing รองรับทิศทางเดียว"

ความรู้: ทิศเดียวจริง ๆ คือ **pipe ธรรมดา (anonymous)** — สองทิศต้อง 2 pipe · Message passing แต่ละ process มีทั้ง send/receive = **สองทิศได้**

ข้อ 3 อาเรย์ int 2 ตัว

"พารามิเตอร์ของ pipe()"

ความรู้: `int pfd[2]; pipe(pfd);` — OS ใส่ค่า 2 ช่อง: **pfd[0] = ปลายอ่าน, pfd[1] = ปลายเขียน** (file (ไฟล์) descriptor ไม่ใช่ชื่อไฟล์ — pipe ธรรมดาไม่มีชื่อ)

ข้อ 4 write(pfd[1], data, length)

"ประโยคส่งข้อมูลเข้า pipe"

ความรู้: pipe ใช้ system call ไฟล์มาตรฐาน: **write()** ส่ง / **read()** รับ · เขียนต้องลงปลายเขียน **pfd[1]** · จำ: "write to 1, read from 0"

ข้อ 5 dup2(pfd[1], 1)

"เปลี่ยนทิศ stdout ไปปลายเขียน pipe"

ความรู้: `dup2(oldfd, newfd)` = ทำให้ **newfd** ชี้ที่เดียวกับ oldfd · fd 0=stdin, 1=stdout, 2=stderr → stdout(1) ไหลเข้าปลายเขียน = `dup2(pfd[1],1)` · pattern มาตรฐาน: ฝั่งเขียน dup2(pfd[1],1) แล้ว exec / ฝั่งอ่าน dup2(pfd[0],0) · ปิดปลายที่ไม่ใช้ก่อน

ข้อ 6 shmget()

"System call ขอสร้าง shared memory"

ความรู้: shm**get** = get — ขอสร้าง/อ้างถึง segment ได้ **shmid** · (shmat=attach, shmdt=detach, shmctl=control/ลบ)

ข้อ 7 ftok()

"ฟังก์ชันสร้าง key ของ shared memory"

ความรู้: `key_t ftok(path, id)` — path + id เดียวกัน → key เดียวกัน → อ้าง shm ก้อนเดียวกันได้ (นิยม path = ".")

ข้อ 8 0764

"rwxrw-r-- เป็นเลขฐานแปดอะไร"

ความรู้: r=4 w=2 x=1 → rwx=7, rw-=6, r--=4 → **0764** (ใช้กับ shmget flag เช่น IPC_CREAT | 0666 ด้วย)

ข้อ 9 IPC_PRIVATE

"shmget ที่การันตีไม่ซ้ำกับใครแน่นอน"

ความรู้: IPC_PRIVATE = key พิเศษ OS สร้าง segment ใหม่ไม่ซ้ำใครเสมอ — นิยมใช้ parent→child (shmid ส่งต่อได้ในตระกูล) · IPC_EXCL แค่ flag กันชน ไม่การันตี

ข้อ 10 shared memory id

"shmget() คืนอะไร"

ความรู้: คืน **shmid** (เลขจำนวนเต็ม) — เอาไปใช้กับ shmat/shmctl/shmdt · flow: key → shmget → **shmid** → shmat → pointer

ข้อ 11 shmat()

"ฟังก์ชันให้ตัวแปรอ้างถึงตำแหน่ง shared memory"

ความรู้: shm**at** = attach — ผูก segment เข้า address space แล้ว **คืน pointer**: `shm_ptr = shmat(shmid, NULL, 0)`

ข้อ 12 False

"ตัวแปรเก็บตำแหน่ง shm ต้องเป็น pointer ไป int เท่านั้น"

ความรู้: shmat คืน **void*** — cast เป็นชนิดไหนก็ได้ (char*, struct data* ...) ข้อจำกัดเดียวคือต้องเป็น pointer

ข้อ 13 struct data *shm_ptr;

"shmget(..., sizeof(struct data), ...) ตัวแปรชี้ต้องประกาศยังไง"

ความรู้: จองขนาด sizeof(struct data) → pointer ต้องตรงชนิด: `struct data *p; p = (struct data *) shmat(shmid, NULL, 0);` ใช้ผ่าน `p->field`

🟢 Shared Memory ครบวงจร (จำ 6 ขั้น)

```c
key_t key = ftok(".", 'a');                    // 1. สร้าง key
int shmid = shmget(key, sizeof(struct data), IPC_CREAT | 0666); // 2. สร้าง segment
struct data *p = (struct data *) shmat(shmid, NULL, 0);   // 3. attach
/* 4. อ่าน/เขียนผ่าน p */
shmdt(p);                                      // 5. detach
shmctl(shmid, IPC_RMID, NULL);                 // 6. ลบ (คนสุดท้าย)
```

K3. Quiz 3 — Semaphore

ข้อ 2 False

"Semaphore ที่ใช้กับ critical section คือ counting"

ความรู้: CS ใช้ **binary semaphore (mutex)** เริ่ม 1: wait→CS→signal · counting (เริ่ม = จำนวน resource) ใช้กับ resource หลายหน่วย เช่น bounded buffer

ข้อ 3 False

"Semaphore การันตีผ่านเงื่อนไข 3 ข้อของ CS"

ความรู้: 3 เงื่อนไข = Mutual Exclusion / Progress / Bounded Waiting — semaphore การันตีได้แค่ **ข้อ 1** · progress/bounded waiting ขึ้นกับ implementation (classic อาจ starvation)

ข้อ 4 2 ตัว

"bounded buffer ใช้ counting semaphore กี่ตัว"

ความรู้: ใช้รวม 3 ตัว: empty(n), full(0), mutex(1) — แต่**counting จริงมี 2** (empty, full) เพราะ mutex เป็น binary

ข้อ 5 empty

"producer ตรวจช่องว่างด้วยตัวไหน"

ความรู้: Producer: `wait(empty)` ขอช่องว่างก่อน · Consumer: `wait(full)` ก่อนอ่าน · mutex ใช้ตอนแตะ buffer จริง (ลำดับ: resource ก่อน mutex!)

ข้อ 6 mutex

"readers-writers คุมจำนวน reader ด้วย semaphore ไหน"

ความรู้: mutex ครอบช่วง readcount++/-- · **readcount เป็น int ธรรมดา** ไม่ใช่ semaphore · wrt คุม writer / reader แรกล็อก-ตัวสุดท้ายปลด

ข้อ 7 resource

"ตะเกียบใน Dining philosopher แทนอะไร"

ความรู้: ตะเกียบ = **shared resource** (นักปราชญ์ = process) · implement ใช้ semaphore 1 ตัวต่อ 1 ตะเกียบ แต่ตัวตนของมันคือ resource

ข้อ 8 False

"semaphore การันตีนักปราชญ์ทุกคนได้กิน"

ความรู้: หยิบซ้ายพร้อมกัน → รอขวาเป็นวง = **deadlock** + ไม่มี bounded waiting → starvation · แก้: จำกัด ≤ 4 คน / คนนึงหยิบขวาก่อน / ใช้ Monitor

ข้อ 9 sem_post()

"PThread semaphore ตัวไหน = signal()"

ความรู้: PThread: `sem_init(&s,0,val)` สร้าง · **sem_wait = P (ลด)** · **sem_post = V (เพิ่ม/ปลุก)** · sem_destroy เก็บกวาด · **ไม่มี sem_signal ใน POSIX!**

K4. Quiz 4 — Introduction to File Management

ข้อ 2 True

"เข้าถึงไฟล์เหมือนกันไม่ว่าอุปกรณ์อะไร"

ความรู้: ระบบไฟล์ = ชั้น abstraction → open/read/write ชุดเดียวกันทุกอุปกรณ์ (**device independence**)

ข้อ 3 False

"Journaling = การเข้ารหัสไฟล์"

ความรู้: Journaling = write-ahead log เพื่อ**กู้คืนหลัง crash** · การเข้ารหัสเป็นฟีเจอร์แยก (NTFS ใช้ **EFS**)

ข้อ 4 ExFAT

"ระบบไฟล์สำหรับ USB drive"

ความรู้: USB/external = **ExFAT** (ใหญ่กว่า 4GB ข้าม Windows-macOS) · NTFS=Windows, ext4=Linux, APFS=Apple

ข้อ 5 False

"contiguous ปัญหาคือ internal fragmentation"

ความรู้: contiguous → **external** fragmentation (ต้องกันบล็อกติดกัน พื้นที่แตกเป็นหลุม) · internal = เศษในบล็อกสุดท้ายที่ไม่เต็ม

ข้อ 6 False

"linked allocation เป็นการจัดสรรแบบต่อเนื่อง"

ความรู้: linked = บล็อกกระจาย + pointer โซ่ = **non-contiguous** โดยนิยาม (FAT อยู่ตระกูลนี้)

ข้อ 7 False

"บล็อกใหญ่ → external fragmentation"

ความรู้: บล็อกใหญ่ → เศษในบล็อกสุดท้ายเยอะ = **internal fragmentation** (ไฟล์ 1B ในบล็อก 4KB = เสีย ~4KB)

ข้อ 8 False (ตามคีย์!)

"single-level index: ขนาดไฟล์สูงสุดถูกจำกัดด้วยจำนวนช่องใน index block"

ความรู้: ⚠️**คีย์ผู้สอน = False** (แม้ตามตำราน่าจะ True) — ตอบ False ตามคีย์ · ความรู้จริง: ไฟล์ใหญ่สุด = entries × block size

ข้อ 9 512

"บล็อก 2KB, pointer 4B → 1 index block มีกี่ช่อง"

ความรู้: entries = 2048 ÷ 4 = **512** → single-level ใหญ่สุด 512 × 2KB = 1MB · สูตรเดียวกับ inode ของ J3.1

ข้อ 10 False

"Symbolic link เรียกอีกอย่างว่า hard link"

ความรู้: symbolic = **soft link** (ไฟล์แยกเก็บ path) · hard link = entry ชี้ inode เดียวกัน

ข้อ 11 True

"Hard link จัดการด้วย Reference Counter"

ความรู้: inode เก็บ **link count** — rm แค่ลด counter ปล่อยพื้นที่เมื่อ = 0

ข้อ 12 False

"คำสั่ง hardln ใช้สร้าง Hard Link"

ความรู้: ไม่มี hardln! จริงคือ `ln target link` (hard) / `ln -s target link` (symbolic)

ข้อ 13 False

"mount อุปกรณ์เดียวกันได้แค่ 1 mount point"

ความรู้: mount อุปกรณ์เดียวได้**หลาย mount point** (รองรับ bind mount) — "ไม่สามารถ" เป็นเท็จ